

SAAS FACTORY V4

Metodologia de Desarrollo

Guia completa del flujo de desarrollo,
despliegue e instalacion para clientes

Agent-First Software Factory

Version 4.0

Abril 2026

Documento Confidencial

Indice de Contenido

1.	Filosofia Agent-First	3
2.	Stack Tecnologico (Golden Path)	4
3.	Arquitectura Feature-First	5
4.	Flujo de Desarrollo	6
5.	Entornos: Local, Staging, Produccion	7
6.	Flujo de Trabajo con Supabase	8
7.	Flujo de Deploy con Vercel	9
8.	Instalacion para Cliente Nuevo	10
9.	Skills: Herramientas del Agente	11
10.	Reglas deCodigo y Convenciones	13
11.	Auto-Blindaje (Mejora Continua)	14
12.	Checklist de Calidad	15

1

Filosofia Agent-First

SaaS Factory V4 opera bajo el paradigma Agent-First: el humano dice QUE quiere construir y el agente decide COMO hacerlo. El usuario no necesita conocimientos tecnicos. El agente traduce lenguaje natural a codigo funcional.

Principios Fundamentales

- El usuario habla en lenguaje natural, el agente ejecuta
- NUNCA se le pide al usuario que ejecute comandos
- NUNCA se le pide al usuario que edite archivos
- El agente hace TODO, el usuario solo aprueba
- Cada error se documenta y nunca ocurre dos veces (Auto-Blindaje)

Flujo de Comunicacion

Usuario: "Quiero una app para gestionar inventario"
Agente: Ejecuta entrevista de negocio -> Genera
BUSINESS_LOGIC.md -> Pregunta diseno -> Implementa todo el
sistema completo

Este enfoque permite que personas sin conocimiento tecnico puedan dirigir el desarrollo de software complejo, manteniendo la calidad y las mejores practicas de ingenieria.

Beneficios del Enfoque

- Velocidad: Features complejas en minutos, no dias
- Consistencia: Mismo stack, mismos patrones, misma calidad siempre
- Escalabilidad: Cada proyecto hereda las mejoras de la fabrica
- Accesibilidad: No se requiere equipo tecnico para operar

2

Stack Tecnologico (Golden Path)

No se ofrecen opciones tecnologicas. Se ejecuta un stack unico, probado y optimizado. Esto elimina la "paralisis por analisis" y garantiza compatibilidad total entre componentes.

Capa	Tecnologia	Proposito
Framework	Next.js 16	App Router, SSR, API Routes,
UI	React 19	Componentes, Server Components,
Lenguaje	TypeScript 5.9	Tipado estatico, seguridad en
Estilos	Tailwind CSS 3.4	Utility-first, responsive, temas
Base de Datos	Supabase	PostgreSQL, Auth, RLS, Storage,
Estado	Zustand 5.0	State management ligero con
Validacion	Zod	Schemas, validacion runtime, type
AI Engine	Vercel AI SDK v5	Chat, RAG, Vision, Tools, Streaming
AI Router	OpenRouter	Multi-modelo: GPT, Claude, Gemini,
Testing	Playwright	E2E automatizado, CLI + MCP
Deploy	Vercel	CI/CD automatico, Edge, Serverless
Email	Nodemailer	SMTP, templates, adjuntos PDF
PDF/Excel	jsPDF + XLSX	Generacion de reportes exportables

IMPORTANTE: Este stack es fijo. No se negocia ni se cambia por proyecto. La consistencia es lo que permite velocidad y calidad predecible.

3

Arquitectura Feature-First

Cada feature contiene TODO su contexto en un solo directorio. Esto facilita el desarrollo, testing y mantenimiento. No hay que buscar en multiples carpetas para entender una funcionalidad.

Estructura del Proyecto

```
src/
|-- app/
|   |-- (auth)/           // Rutas de autenticacion (login, signup)
|   |-- (main)/           // Rutas principales con layout sidebar
|   |-- api/              // API Routes (CRUD, emails)
|-- features/
|   |-- [feature]/
|       |-- components/   // UI especifica de la feature
|       |-- hooks/        // Logica y estado local
|       |-- services/     // Llamadas a API/Supabase
|       |-- types/        // Tipos TypeScript
|       |-- store/        // Estado Zustand
|-- shared/
|   |-- components/       // Componentes reutilizables
|   |-- hooks/            // Hooks compartidos
|   |-- lib/              // Utilidades (export, format)
|   |-- types/            // Tipos globales
```

Patron de Datos

- 1 Usuario interactua con la UI (React Components)
- 2 Cambios se guardan en Zustand Store (con persistencia)
- 3 Store sincroniza con Supabase via servicios de la feature
- 4 Supabase aplica RLS (Row Level Security) automaticamente
- 5 Datos fluyen de vuelta a la UI via React Server Components o queries

4

Flujo de Desarrollo

El desarrollo sigue flujos predefinidos segun la complejidad de la tarea. Esto garantiza que cada tipo de trabajo reciba el nivel de planificacion adecuado.

Flujo 1: Proyecto Nuevo (desde cero)

- 1 NEW-APP: Entrevista de negocio que genera BUSINESS_LOGIC.md con toda la logica del sistema
- 2 Seleccion de Design System: Elegir estilo visual (Glassmorphism, Neobrutalism, etc.)
- 3 ADD-LOGIN: Implementar autenticacion completa (Email + Google OAuth + profiles + RLS)
- 4 PRP: Planificar la primera feature con Product Requirements Proposal
- 5 BUCLE AGENTICO: Implementar por fases (BD -> Backend -> Frontend -> Testing)
- 6 QA: Verificar con Playwright que todo funciona correctamente

Flujo 2: Feature Compleja

- 1 PRP: Generar plan detallado (el usuario aprueba antes de implementar)
- 2 BUCLE AGENTICO: Ejecutar por fases con mapeo de contexto real en cada fase
- 3 AUTO-BLINDAJE: Si hay errores, se documentan y corrigen automaticamente
- 4 QA: Validar resultado final con tests automatizados

Flujo 3: Tarea Rapida (Sprint)

- 1 SPRINT: Ejecutar directo sin planificacion formal
- 2 Verificar con el usuario que el resultado es correcto
- 3 Pasar a la siguiente tarea

REGLA: Si la tarea toca mas de 3 archivos o requiere cambios en BD + API + UI, SIEMPRE usar PRP + Bucle Agentic. Nunca improvisar features complejas.

5

Entornos: Local, Staging, Produccion

El desarrollo debe fluir por tres entornos claramente definidos. Cada entorno tiene un proposito especifico y nunca se debe saltar un paso.

Entorno Local (Desarrollo)

- URL: `http://localhost:3000`
- Base de datos: Supabase local (puerto 54321) o Supabase remoto de desarrollo
- Proposito: Desarrollo activo, pruebas rapidas, iteracion
- Comando: `npm run dev` (auto-detecta puerto disponible 3000-3006)

Entorno Staging (Validacion)

- URL: `proyecto.vercel.app` (URL automatica de Vercel)
- Base de datos: Supabase remoto (proyecto de desarrollo/staging)
- Proposito: Validar que todo funcione en entorno real antes de entregar al cliente
- Deploy: Automatico con cada push a la rama principal

Entorno Produccion (Cliente)

- URL: Dominio del cliente o subdominio dedicado
- Base de datos: Supabase remoto (proyecto exclusivo del cliente, BD vacia)
- Proposito: Sistema en uso real por el cliente
- Deploy: Vercel con variables de entorno del cliente

Flujo entre Entornos



REGLA DE ORO: NUNCA instalar directamente en produccion del cliente. Siempre validar en staging (Vercel + Supabase remoto) primero. Lo que funciona en local NO siempre funciona en produccion.

6

Flujo de Trabajo con Supabase

Supabase es la base de datos central del sistema. Provee PostgreSQL, autenticacion, Row Level Security y storage. Cada cliente tiene su propio proyecto de Supabase.

Estructura de Migraciones

Las migraciones SQL viven en /supabase/migrations/ y definen toda la estructura de la base de datos. Se aplican automaticamente al crear un proyecto nuevo.

- Tablas de referencia: tipo_identificacion, categoria, grupo, subgrupo, unidad_medida, etc.
- Tablas de negocio: producto, proveedor, bodega, centro_costo, orden_compra, etc.
- RLS habilitado en TODAS las tablas (obligatorio)
- Extension uuid-ossf para IDs unicos

Row Level Security (RLS)

RLS es OBLIGATORIO en todas las tablas. Controla que cada usuario solo vea y modifique los datos que le corresponden. Sin RLS, cualquier usuario autenticado podria ver todos los datos.

Fase	Politica RLS
Desarrollo	Permisiva (Allow all for authenticated) - temporal
Staging	Granular por rol (admin, supervisor, operador)
Produccion	Granular + auditoria + validacion estricta

Comandos Supabase

- supabase start - Iniciar Supabase local
- supabase db reset - Resetear BD local y aplicar migraciones
- supabase migration new [nombre] - Crear nueva migracion
- supabase db push - Aplicar migraciones a proyecto remoto
- supabase link --project-ref [ID] - Vincular a proyecto remoto

IMPORTANTE: Nunca modificar la BD directamente en produccion. Siempre crear migraciones y aplicarlas en orden. Esto garantiza que la estructura sea reproducible.

7

Flujo de Deploy con Vercel

Vercel es la plataforma de deploy. Cada push al repositorio genera un deploy automatico. Vercel provee URLs de preview para cada branch y una URL de produccion estable.

Configuracion Inicial

- 1 Crear repositorio en GitHub (privado)
- 2 Conectar repositorio a Vercel (vercel.com > Import Project)
- 3 Configurar variables de entorno en Vercel:
 - NEXT_PUBLIC_SUPABASE_URL - URL del proyecto Supabase
 - NEXT_PUBLIC_SUPABASE_ANON_KEY - Clave publica de Supabase
 - NEXT_PUBLIC_SITE_URL - URL del sitio en Vercel
 - SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS - Para emails

Flujo de Deploy

- 1 Desarrollar en local (npm run dev)
- 2 Verificar build (npm run build + npm run typecheck)
- 3 Commit y push a GitHub
- 4 Vercel detecta el push y genera deploy automatico
- 5 Verificar en URL de preview que todo funcione
- 6 Merge a main = deploy a produccion

Variables de Entorno por Entorno

Variable	Desarrollo	Produccion
SUPABASE_URL	localhost:54321	xxx.supabase.co
SITE_URL	localhost:3000	dominio-cliente.com
SMTP_*	Gmail dev	SMTP del cliente

NUNCA subir archivos .env al repositorio. Las variables se configuran directamente en el dashboard de Vercel para cada entorno (Preview, Production).

8

Instalacion para Cliente Nuevo

Este es el proceso completo para instalar el sistema en un cliente nuevo. El sistema se entrega como una aplicacion lista con base de datos vacia que el cliente llena con sus propios datos.

Paso 1: Crear Proyecto Supabase

- Ir a supabase.com y crear nuevo proyecto (plan Free o Pro segun necesidad)
- Anotar: Project URL, Anon Key, Service Role Key
- Region: Seleccionar la mas cercana al cliente

Paso 2: Aplicar Estructura de BD

- Vincular proyecto: `supabase link --project-ref [PROJECT_ID]`
- Aplicar migraciones: `supabase db push`
- Verificar tablas creadas en el dashboard de Supabase
- Configurar RLS granular (reemplazar politicas permisivas)

Paso 3: Configurar Autenticacion

- Habilitar Email/Password en Supabase Auth
- Configurar Google OAuth si el cliente lo requiere
- Configurar SMTP para emails de confirmacion
- Crear usuario administrador inicial

Paso 4: Deploy en Vercel

- Crear nuevo proyecto en Vercel (o duplicar existente)
- Configurar variables de entorno del cliente
- Asignar dominio personalizado si aplica
- Verificar que el deploy funcione correctamente

Paso 5: Entrega al Cliente

- Entregar credenciales de administrador
- Capacitar en uso basico del sistema
- Verificar que puede crear datos (productos, proveedores, etc.)
- Soporte inicial durante primera semana

TIEMPO ESTIMADO: La instalacion para un cliente nuevo toma aproximadamente 1-2 horas si el sistema ya esta validado en staging. La mayor parte del tiempo es configuracion, no codigo.

9

Skills: Herramientas del Agente

Los Skills son herramientas especializadas que el agente utiliza para ejecutar tareas. Se dividen en dos categorias: los que el usuario puede solicitar y los que el agente activa automaticamente.

Skills que el Usuario Puede Pedir

Skill	Cuando Usarlo
new-app	Empezar proyecto desde cero. Entrevista de negocio -> BUSINESS_LOGIC.md
add-login	Agregar auth completa: Email + Google OAuth + profiles + RLS
landing	Landing page cinematografica: scroll-driven video + copy de alta conversion
prp	Planificar feature compleja antes de implementar
bucle-agentico	Implementar features complejas por fases coordinadas
sprint	Tareas rapidas que no necesitan planificacion formal
ai	Agregar IA: chat, RAG, vision, tools, web search
qa	Testing automatizado con Playwright CLI
memory-manager	Guardar conocimiento persistente del proyecto
image-generation	Generar imagenes con OpenRouter + Gemini
eject-sf	Remover SaaS Factory del proyecto (DESTRUCTIVO)
update-sf	Actualizar template a ultima version
skill-creator	Crear nuevos skills personalizados
autoresearch	Auto-optimizar skills con loop autonomo

Skills Automaticos (el usuario no los invoca)

Skill	Se Activa Cuando...
backend	Server Actions, APIs, logica de negocio, Zod
frontend	UI/UX, componentes React, Tailwind, animaciones
supabase-admin	Migraciones, RLS, queries SQL, config auth
codebase-analyst	Entender patrones y arquitectura del proyecto
vercel-deployer	Deploy, env vars, dominios, rollbacks
documentacion	Actualizar docs despues de cambios
calidad	Testing, validacion, quality gates

MCPs (Model Context Protocol)

Los MCPs son conexiones directas a servicios externos que el agente usa como "sentidos y manos":

- Playwright MCP: Ver la UI, hacer click, llenar formularios, tomar screenshots
- Supabase MCP: Ejecutar SQL, aplicar migraciones, listar tablas, ver logs
- Next.js DevTools MCP: Ver errores de build/runtime en tiempo real

10

Reglas deCodigo y Convenciones

Estas reglas son obligatorias en todo el codigo. No son sugerencias. El agente las aplica automaticamente y el Auto-Blindaje las refuerza.

Principios de Diseno

- KISS (Keep It Simple, Stupid): Soluciones simples siempre
- YAGNI (You Ain't Gonna Need It): Solo implementar lo necesario
- DRY (Don't Repeat Yourself): Sin duplicacion de codigo

Limites de Tamano

Elemento	Limite Maximo
Archivos	500 lineas
Funciones	50 lineas
Componentes	300 lineas (incluyendo JSX)
Props	10 propiedades maximo

Convenciones de Nombrado

Elemento	Convencion	Ejemplo
Variables	camelCase	totalProductos, isLoading
Funciones	camelCase	handleSubmit, fetchData
Componentes	PascalCase	ProductoForm, DashboardCard
Archivos	kebab-case	producto-form.tsx, use-auth.ts
Tipos	PascalCase	Producto, OrdenCompra
Constantes	UPPER_SNAKE	MAX_ITEMS, API_URL

Reglas de Seguridad

- NUNCA usar "any" en TypeScript (usar "unknown" si es necesario)
- SIEMPRE validar entradas de usuario con Zod
- SIEMPRE habilitar RLS en tablas de Supabase
- NUNCA exponer secrets en codigo o en NEXT_PUBLIC_*
- NUNCA almacenar tokens de sesion en localStorage sin encriptacion

11

Auto-Blindaje (Mejora Continua)

El Auto-Blindaje es el sistema de mejora continua de SaaS Factory. Cada error que ocurre se arregla, se documenta y se previene. El mismo error NUNCA ocurre dos veces.

Ciclo de Auto-Blindaje

- 1 ERROR OCURRE: Se detecta un fallo durante desarrollo o testing
- 2 SE ARREGLA: El agente corrige el error inmediatamente
- 3 SE DOCUMENTA: Se registra la causa raiz y la solucion
- 4 SE PREVIENE: Se actualiza el skill o regla para evitar recurrencia

Donde se Documenta Cada Error

Ubicacion	Tipo de Error
PRP actual	Errores especificos de la feature en desarrollo
Skill relevante	Errores que aplican a multiples features
CLAUDE.md	Errores criticos que aplican a TODO el sistema
Memory (.claude/)	Aprendizajes del proyecto especifico

Ejemplo Real de Auto-Blindaje

2025-01-09: Usar npm run dev, no next dev
- Error: Puerto hardcoded causa conflictos cuando multiples proyectos corren
- Fix: Siempre usar npm run dev que auto-detecta puerto disponible (3000-3006)
- Aplicar en: TODOS los proyectos de la fabrica

Este sistema convierte cada error en una mejora permanente. Con el tiempo, la fabrica se vuelve mas robusta y los errores comunes desaparecen completamente.

12

Checklist de Calidad

Antes de entregar cualquier feature o sistema a un cliente, verificar TODOS los puntos de esta checklist. No se hace excepcion.

Antes de Cada Deploy

- ☐ npm run build completa sin errores
- ☐ npm run typecheck pasa sin errores
- ☐ npm run lint pasa sin warnings criticos
- ☐ Variables de entorno configuradas correctamente en Vercel
- ☐ Migraciones SQL aplicadas en la BD de destino
- ☐ RLS configurado y probado en todas las tablas

Antes de Entregar al Cliente

- ☐ Sistema accesible via URL de produccion
- ☐ Login funciona correctamente (email + password)
- ☐ Usuario administrador creado con credenciales seguras
- ☐ Todos los modulos activos y funcionales
- ☐ Emails de notificacion configurados y probados
- ☐ Exportacion PDF/Excel funciona en todos los reportes
- ☐ Datos de prueba eliminados (BD limpia)
- ☐ Dominio personalizado configurado (si aplica)
- ☐ SSL/HTTPS activo y funcionando
- ☐ Backup de BD configurado (si aplica)

Validacion Funcional Minima

- ☐ Crear, editar y eliminar un producto
- ☐ Crear un proveedor y asociarlo a un producto
- ☐ Generar una orden de compra con detalle
- ☐ Registrar recepcion de factura
- ☐ Transferir producto entre bodegas
- ☐ Generar reporte y exportar a PDF/Excel
- ☐ Enviar email de orden de compra



Verificar dashboard con datos reales

SaaS Factory V4

Metodologia de Desarrollo

El humano dice QUE.

El agente decide COMO.

Todo es un Skill. Agent-First.

Version 4.0 - Abril 2026

Documento generado automaticamente por SaaS Factory